3A. ADDERS PART 1

BINARY LONG ADDITION

Position		4	3	2	1	0
Carries		1	1	0	0	
Operand1		0	1	1	1	0
Operand2		1	1	1	0	0
Sum	1	0	1	0	1	0

OBSERVATIONS ON WORDLENGTH

- Wordlength is increased by 1 bit due to the final carry out
  - But only if operands are the same length
- Successive additions not necessarily add a bit

Position	4	3	2	1	0
Carries	1	1	0	0	
Operand1	0	1	1	1	0
Operand2	1	1	1	0	0
Sum	1	0	1	0	1	0

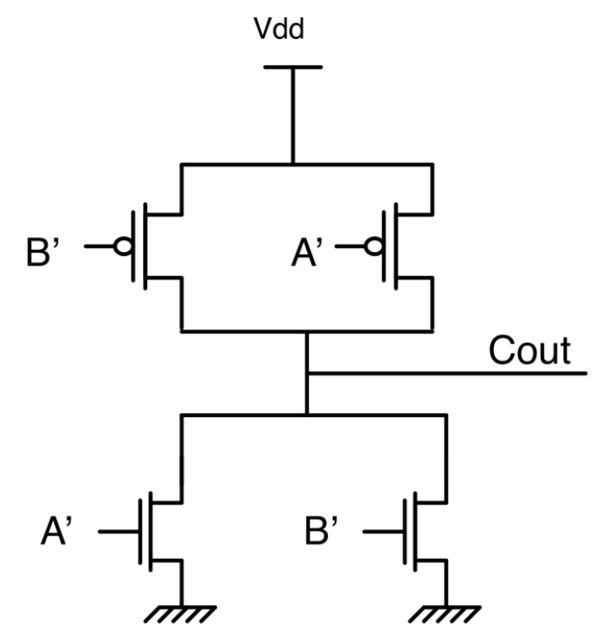
OBSERVATIONS ON OPERATIONS

- In the first bit position only the two operand bits are added
  - Yielding two result bits, sum and carry out
- In all other positions three bits are added:
  - Two operands and one carry in
  - Producing one sum bit and one carry out

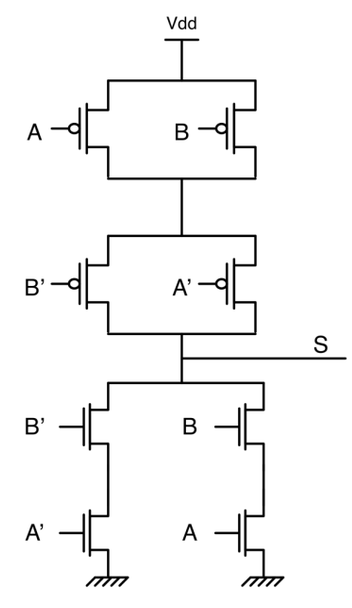
Position	4	3	2	1	0
Carries	1	1	0	0	
Operand1	0	1	1	1	0
Operand2	1	1	1	0	0
Sum	1	0	1	0	1	0

HALF ADDER TRUTH TABLE AND CIRCUITS

- Cout is AND, S is XOR
- HA is of negligible importance
  - Even for first bit position assume a carry in from the outside



A	B	Cout	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



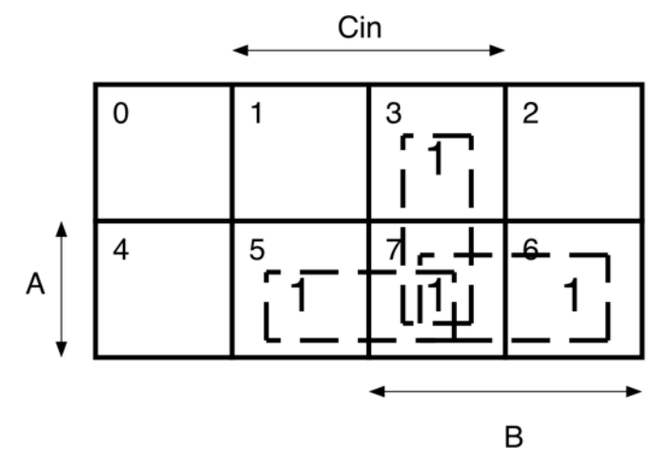
FULL ADDER TRUTH TABLE

- The full adder is the principle functional unit of all arithmetic
- It is actually two functions: sum and carry out
  - We will always ask the question which matters more

A	B	Cin	Cout	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

FULL ADDER LOGIC FUNCTIONS

- Cout can be simplified using K-maps
- S is stuck at a 3 input XOR
- S is obviously a more complicated circuit
  - S will have more delay
  - Will it be the critical path?



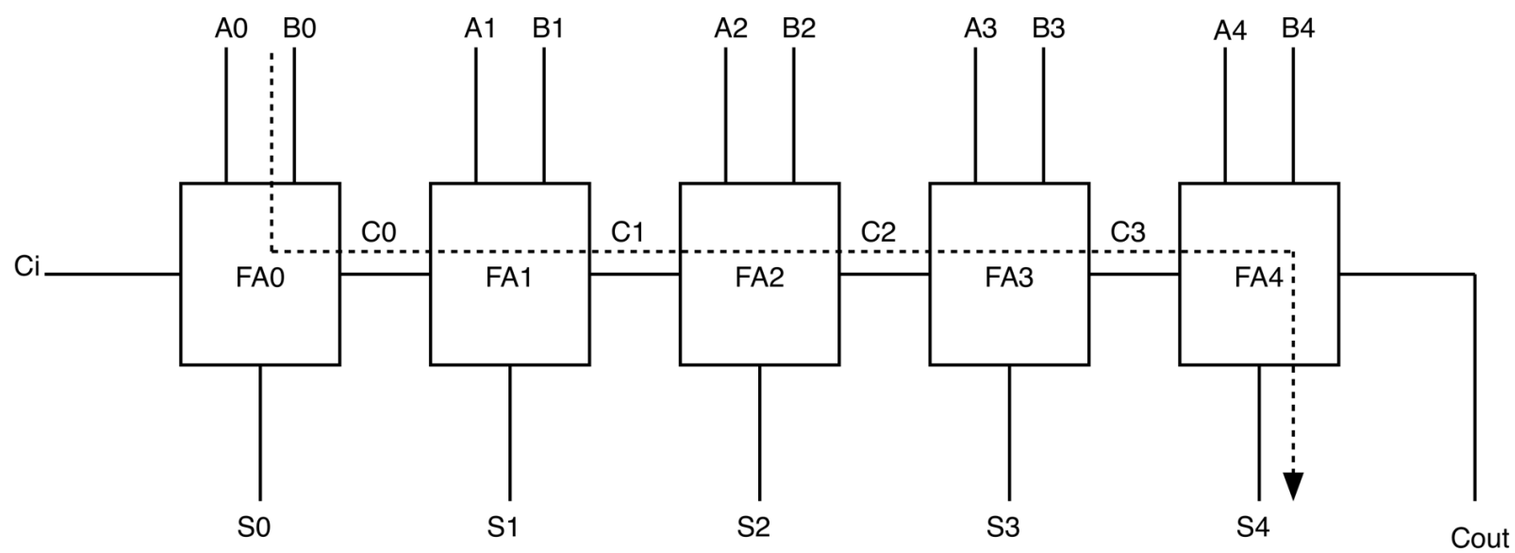
$$S = \overline{A}\,\overline{B}C_{in} + A\overline{B}\,\overline{C}_{in} + \overline{A}B\overline{C}_{in} + ABC_{in}$$
$$C_{out} = AB + AC_{in} + BC_{in}$$

OBSERVATIONS ABOUT THE 4-BIT ADDER

- Each position is a full adder
  - Accepts Bits Ai and Bi
    - Plus Cin from previous position
  - Produces S bit for current position
    - And Cout bit becomes Cir for next position
- Assume first bit also FA
- Last position Cout becomes part of the solution

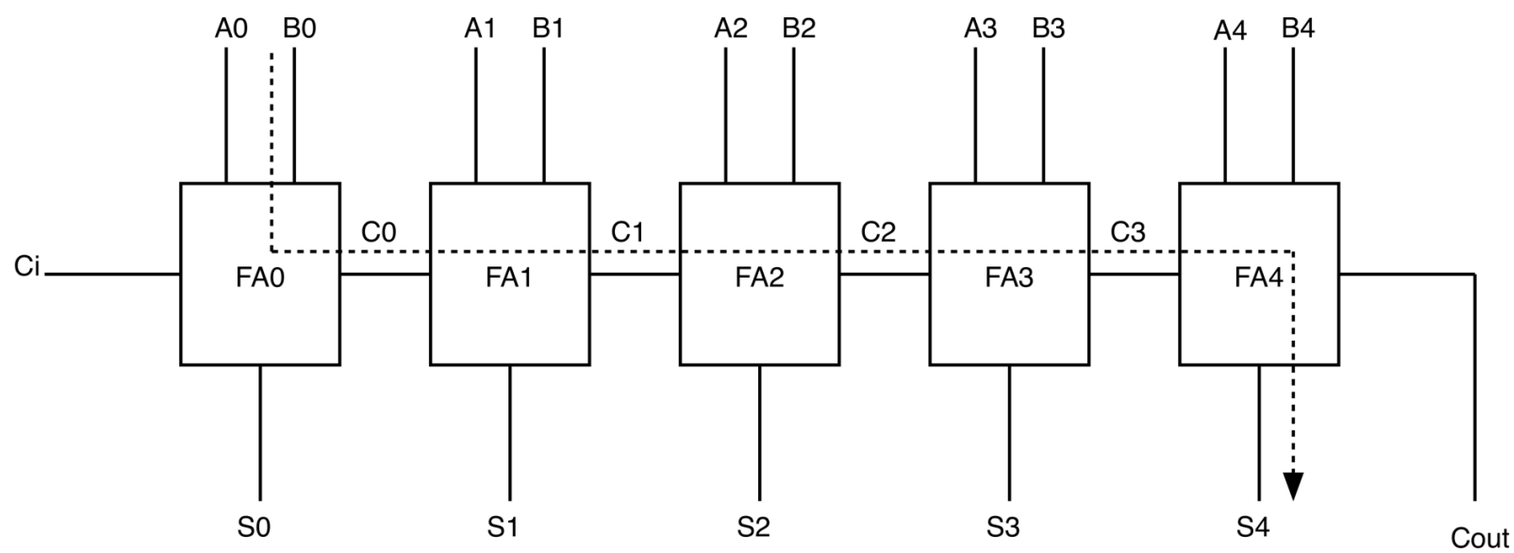
Position	4	3	2	1	0
Carries	1	1	0	0	
Operand1	0	1	1	1	0
Operand2	1	1	1	0	0
Sum	1	0	1	0	1	0

FORMING A 4-BIT ADDER USING FULL ADDERS



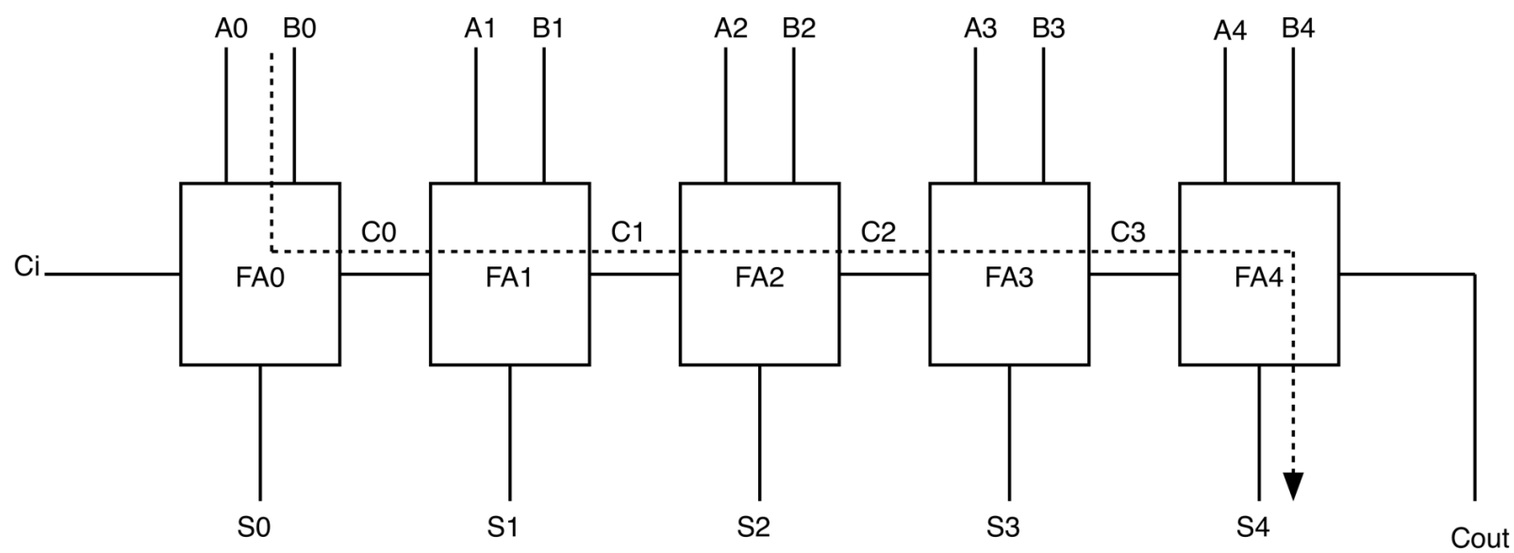
- Each FA feeds the next
- The carry ripples through the adder from Cin to Cout

THINKING ABOUT THE CRITICAL PATH



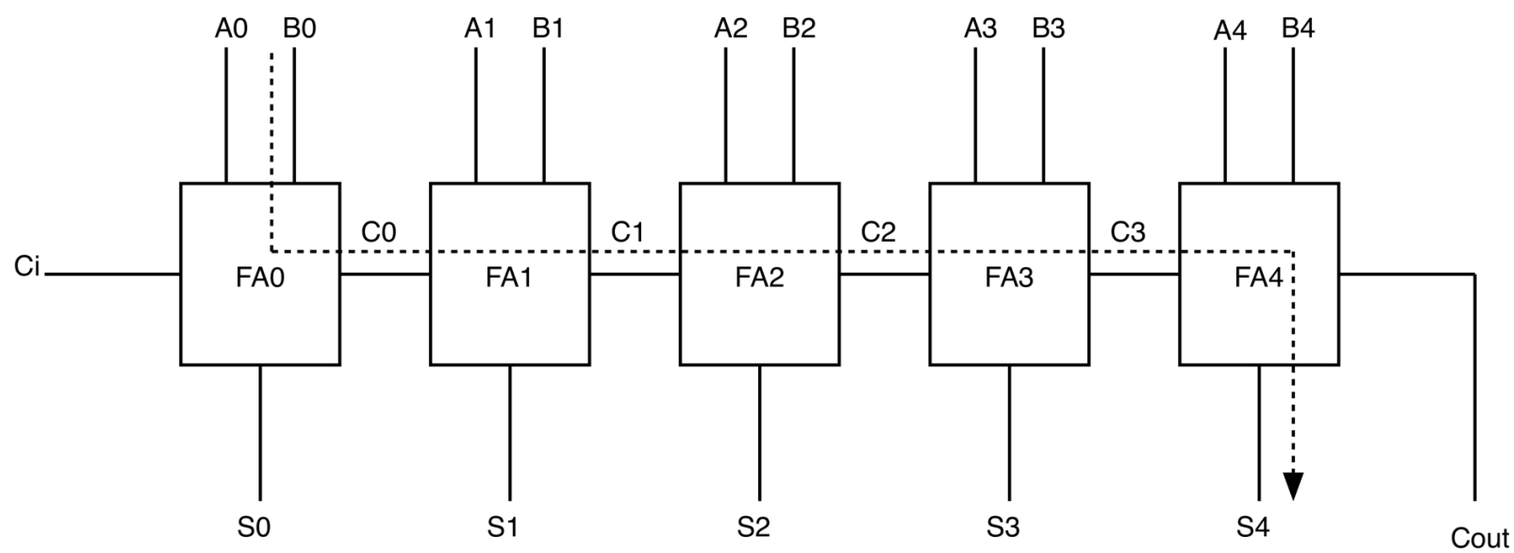
- To calculate Co need Ao, Bo
- To calculate C1 need Co
  - So need to wait for it
- To calculate C2 need C1
  - But C1 needs Co

IDENTIFYING THE CRITICAL PATH



- The adder is done when S0 through Cout are done
- S4 needs C3, which needs C2, which needs C1, etc.
- Critical path starts at A0, B0, Ci, and propagates through all carries and down the last sum
  - Why not the last carry?

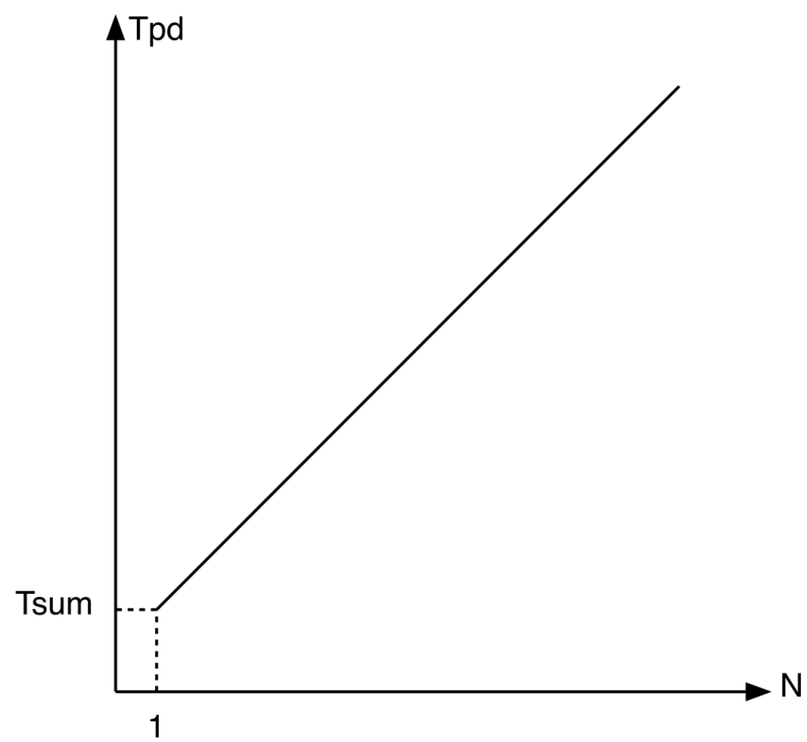
CRITICAL PATH DELAY



- The critical path delays is:
  - 3Tcarry+Tsum
  - In fact: (4-1)Tcarry+Tsum

DEPENDENCIES OF CRITICAL PATH DELAY

- By extension for N-bit adder:
  - (N-1)Tcarry+Tsum



OBSERVATIONS ON CRITICAL PATH DELAY

- Delay is O(N)
- Linear delay dependency on number of bits
  - Tsum is an “intercept” term
  - Tcarry is the slope of the line
- Even though Tsum>Tcarry, carry delay is more important
  - Multiplied by N
  - Appears in the slope

RESTATING THE FULL ADDER

- The Generate-Propagate logic is a way to restate the full adder
- It does not introduce any area savings
- It does not even introduce immediate delay savings
- Its impact is entirely on the carry logic
- However, it prepares us for some of the fastest adders available

THE GENERATE-PROPAGATE TRUTH TABLE

A	B	Cin	Cout	S	G	D	P
0	0	0	0	0	0	1	0
0	0	1	0	1	0	1	0
0	1	0	0	1	0	0	1
0	1	1	1	0	0	0	1
1	0	0	0	1	0	0	1
1	0	1	1	0	0	0	1
1	1	0	1	0	1	0	0
1	1	1	1	1	1	0	0

WHAT IS G-P-D?

- We observe that the state of Cout is divided into three sections
  - The first two rows of the truth table Cout=0 regardless of Cin, Delete
  - The middle four rows, Cout=Cin, Propagate
  - The final two rows Cout=1 regardless of Cin, Generate

WHEN TO G-P-D?

- D for the first two rows
- P for the four middle rows
- G for the bottom two rows
- D when both A and B are 1
  - D=A'B'
- P when either of A and B are 1
  - P=A xor B
- D when both A and B are 1
  - G=AB

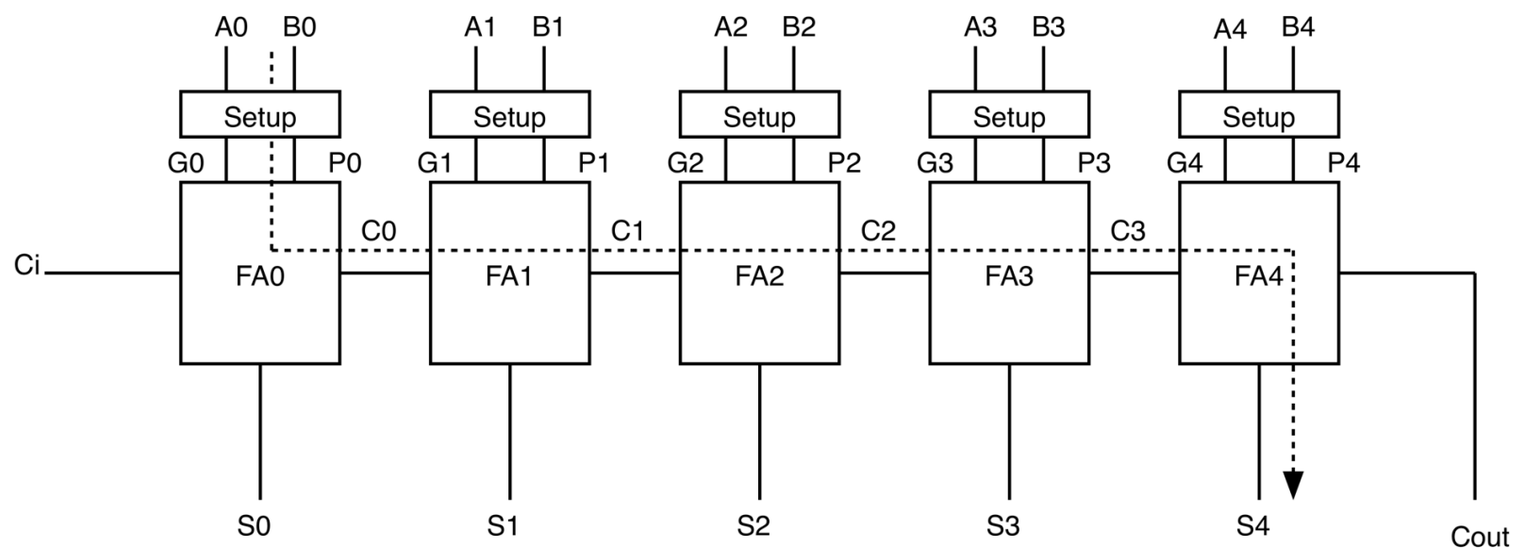
WHAT CHARACTERIZES G-P-D

- G=AB, P=A xor B, D=A'B'
- All can be calculated only from A and B
- Thus, we do not need to wait for any C's to calculate G-P-D at any position
- Thus all G-P-D's can immediately be calculated at the beginning
  - Add a setup delay to beginning
  - Equivalent to A's and B's

COUT IN TERMS OF G-P-D

- Cout is 1 if it is generated or a ‘1’ carry in is propagated
- Cout = G + P Cin
- Delete does not show up because it is a redundant term

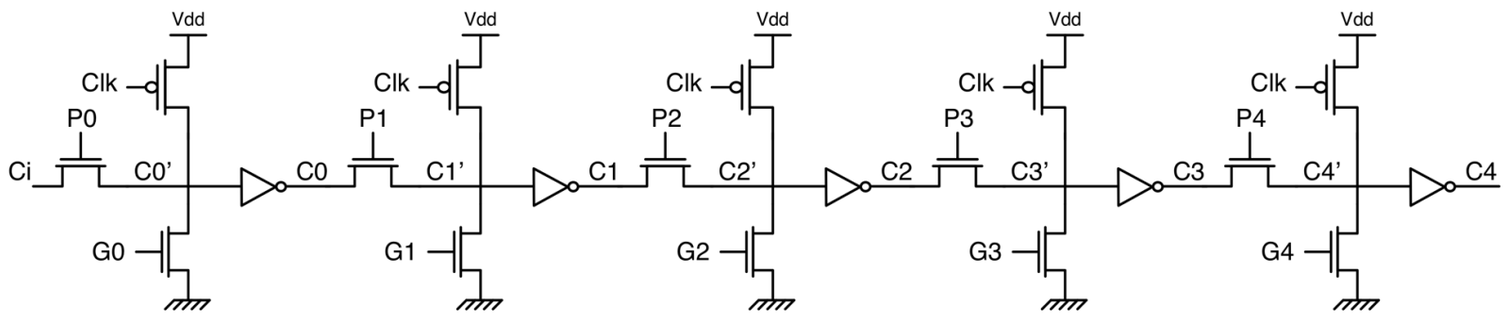
RIPPLE CARRY ADDER WITH G-P LOGIC



$$t_{pd} = t_{gp} + (N-1)t_{carry} + t_{sum}$$

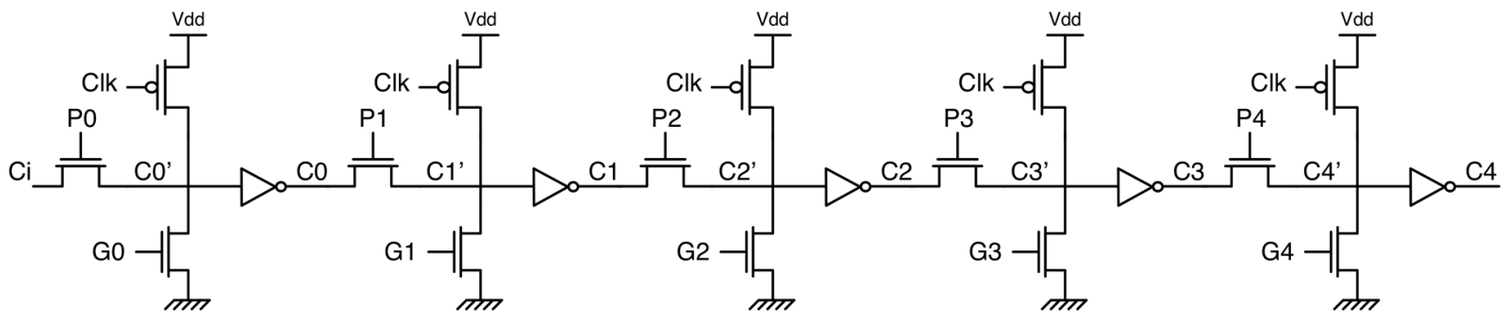
- Identical behavior
- Only an overhead in tgp
  - Appears only once

MANCHESTER CARRY CHAIN



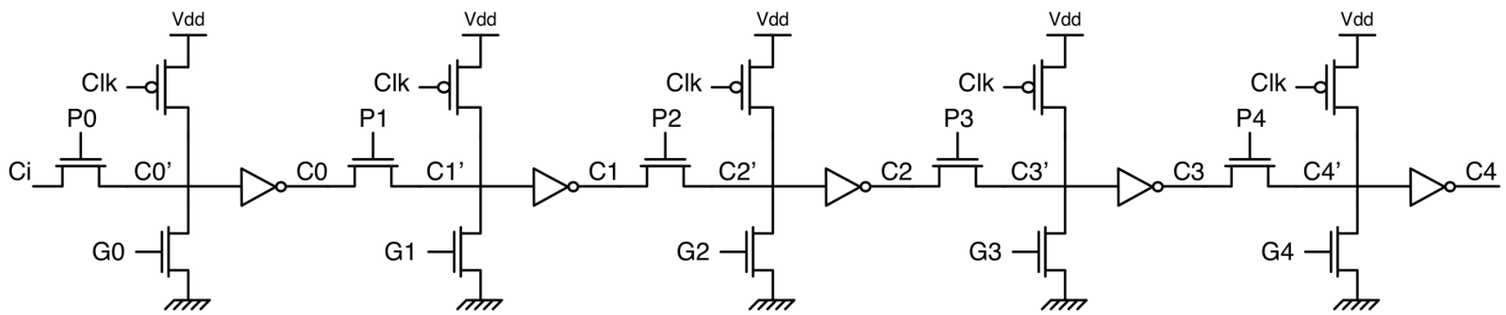
- G-P logic allows us to design excellent carry chains
- Called Manchester carry chain
- Each stage is a domino dynamic gate

MANCHESTER CARRY CHAIN



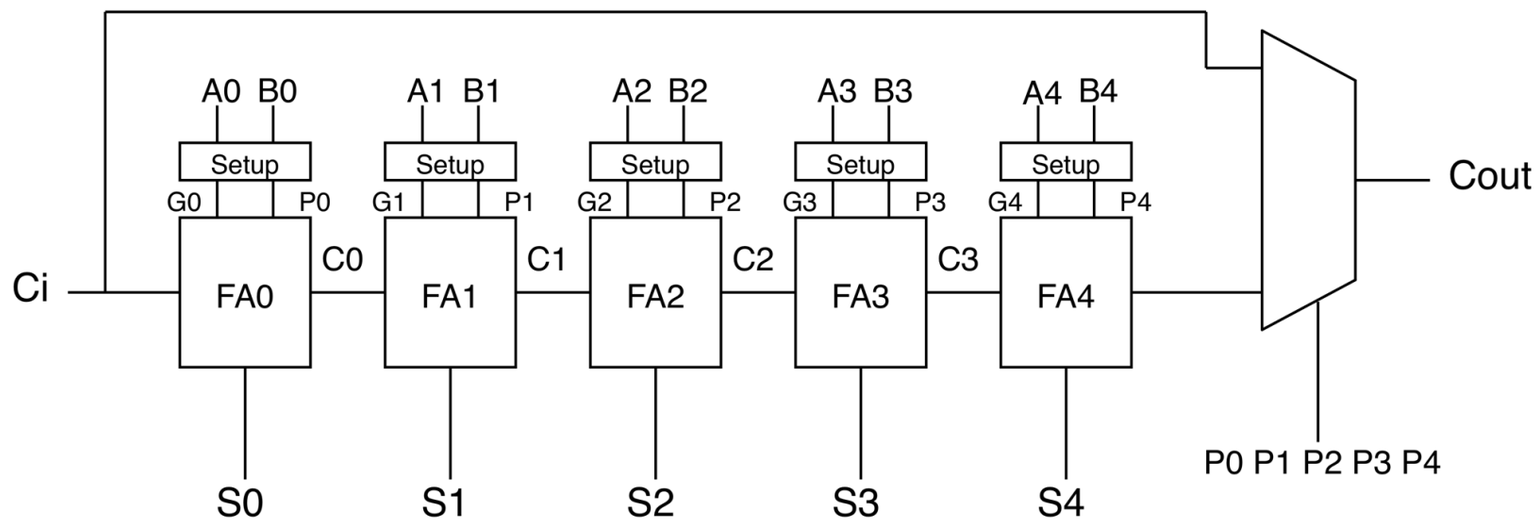
$$PDN = (G + PC_{in})' = G'(P' + C_{in}')$$
$$= G'P' + G'C_{in}'$$

MANCHESTER CARRY CHAIN



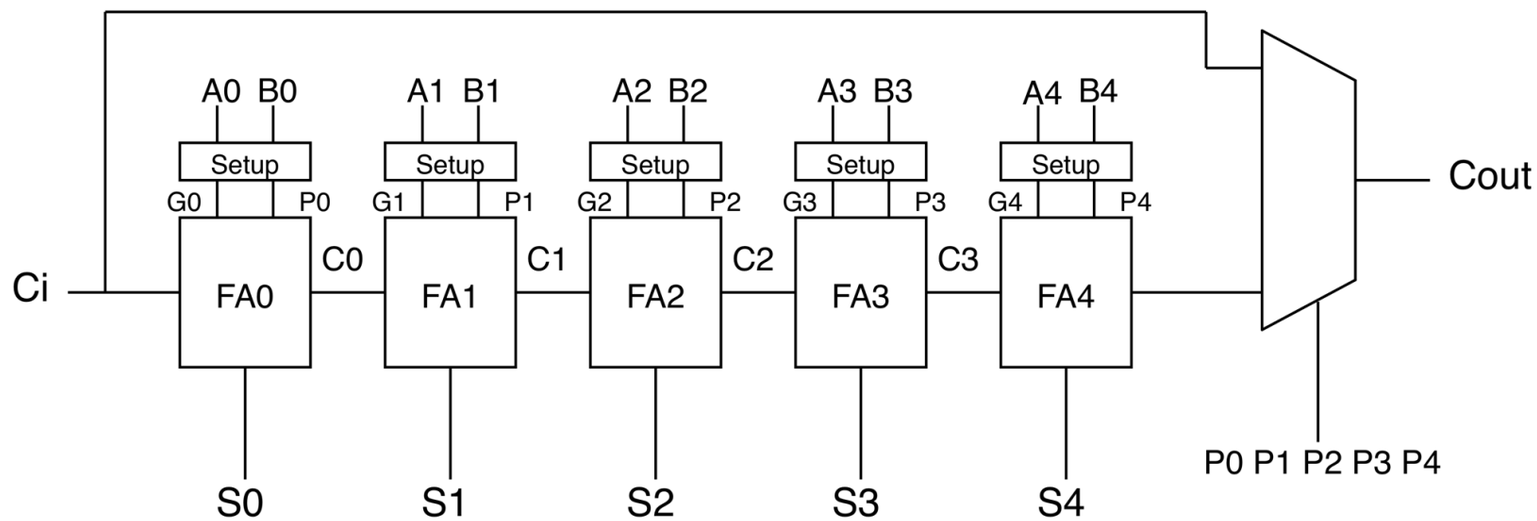
- Area not improved
  - The setup circuits add significant area, brings full adder back to original area
- The static inverters prevent the chain from developing quadratic delay
- Still O(N) delay

AWESOME MODIFIED FIVE BIT ADDER



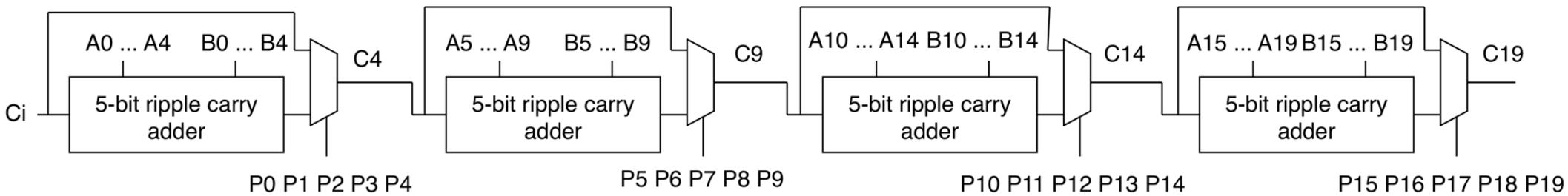
- This adder looks ingenious
- It bypasses the entire carry chain if all bit positions are propagating
- G-P logic has allowed us to create an excellent adder!

AWESOME MODIFIED FIVE BIT ADDER THAT'S TOTALLY USELESS



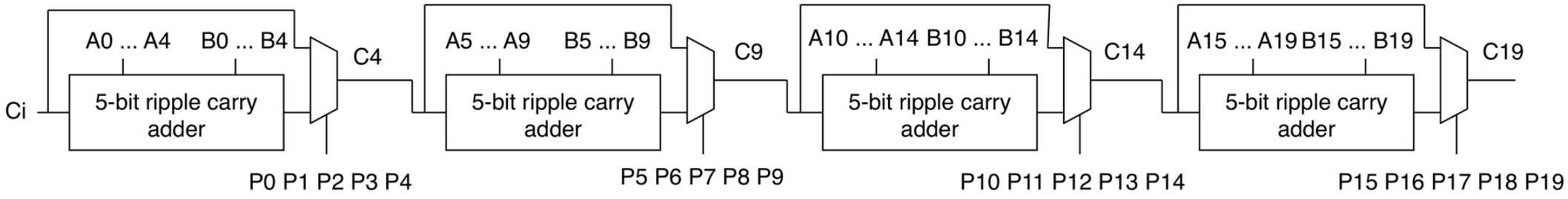
- What is the critical path?
  - Still goes through the entire chain
  - 4Tcarry+Tsum
- Bypass is a best case, which does not affect our clock rate

TRULY GREAT MODIFIED ADDER



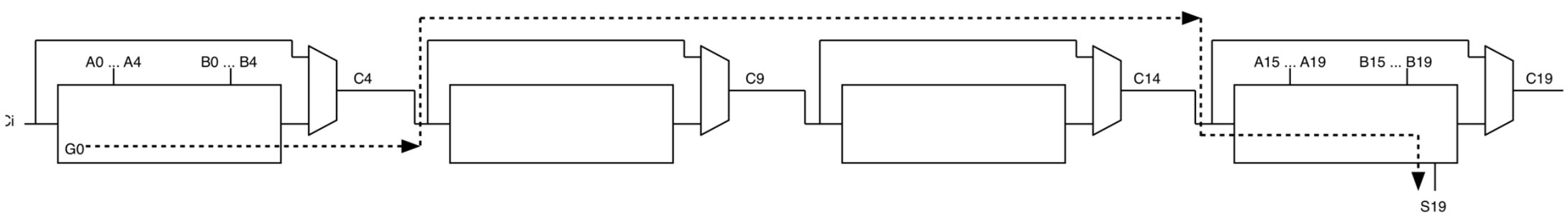
- Bypass works if it is performed on subsets of the adder range
- For example for the 20-bit adder above
  - Divide into four blocks of five bits each
  - Each block can be bypassed

TRULY GREAT MODIFIED ADDER



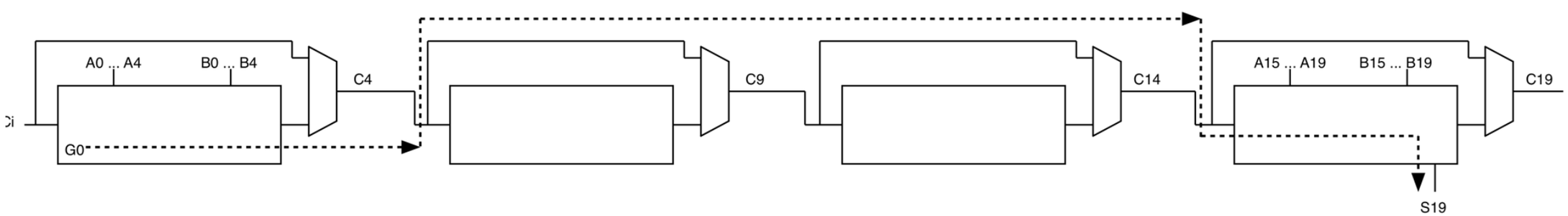
- But what is the critical path?
- This adder's critical path is very challenging to understand
  - Usually understood only through contrarian means

DEFINE THE CRITICAL PATH



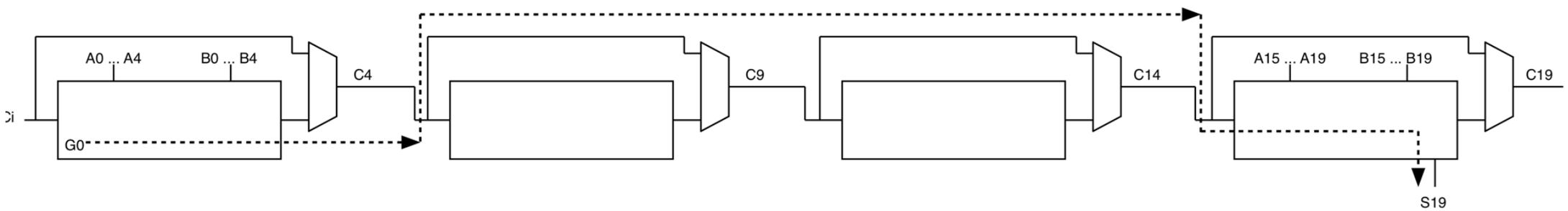
- Critical path goes through the entire first carry chain
- It bypasses all the middle blocks
- It goes through the final carry block, exiting at the final FA

INPUTS FOR CRITICAL PATH ACTIVATION



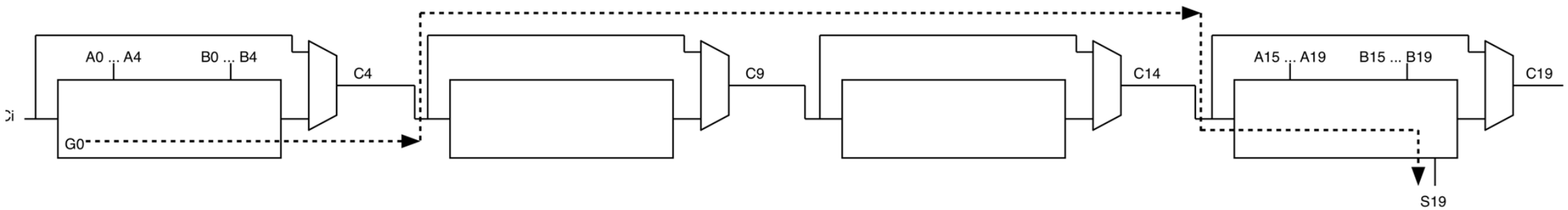
- The critical path is active when position 0 generates
- Every other position (all the way through the rest of the adder) must propagate
  - This includes even the last block

INPUTS FOR CRITICAL PATH ACTIVATION

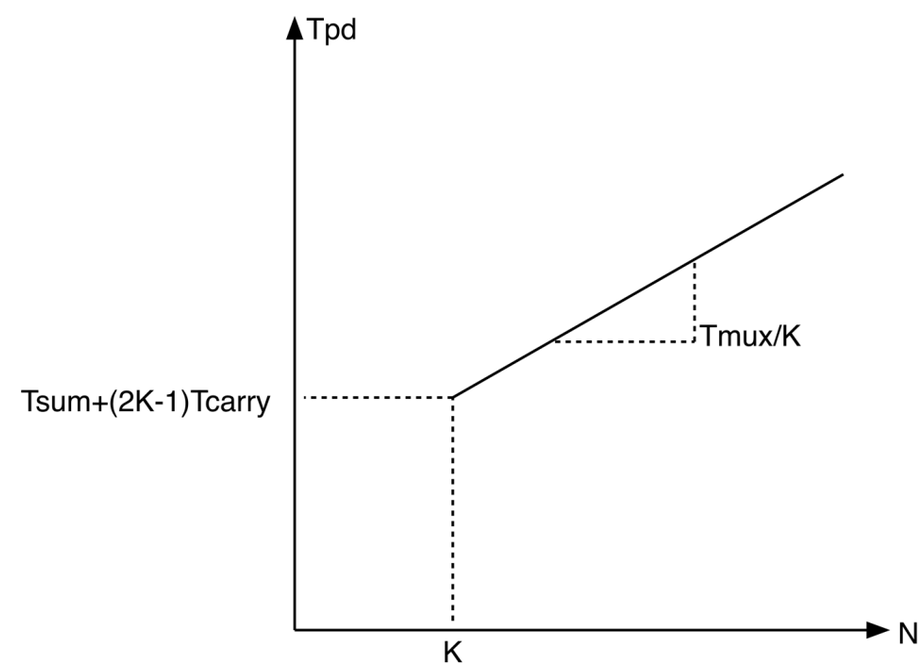


- It seems strange that a case where everything is propagating would cause critical path delay
- But any other input would cause a middle bit position to generate, causing the path to start there, which would be shorter
- In the last block we are waiting for S(N-1), thus must go through the chain regardless

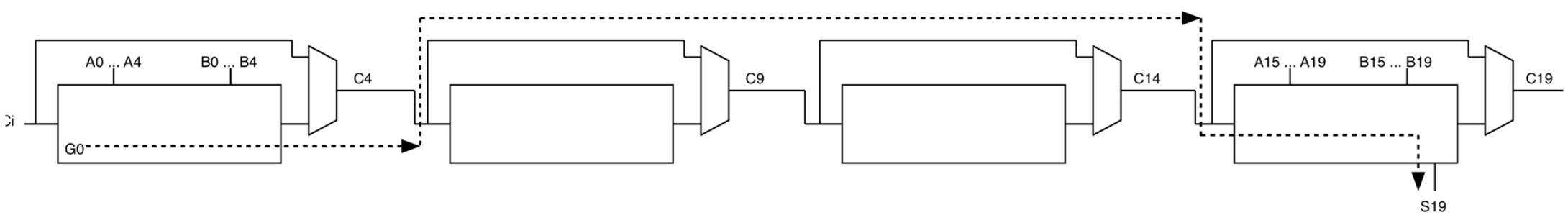
CRITICAL PATH DELAY



$$t_{pd} = Kt_{carry} + (N/K - 1)t_{mux} + (K - 1)t_{carry} + t_{sum}$$



IS THERE AN OPTIMAL VALUE FOR K?



- Increasing K reduces the slope, but increases the intercept
- Thus there is an optimal value for K

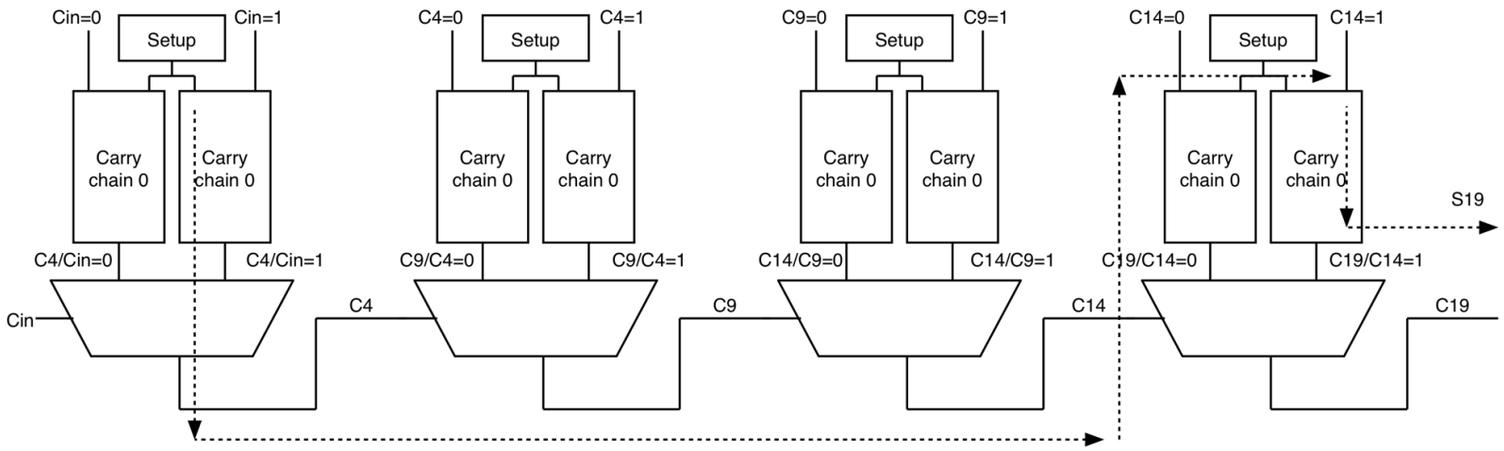
$$\frac{\partial t_{pd}}{\partial K} = t_{carry} - \frac{N}{K^2} t_{mux} + t_{carry} = 0$$

$$K^2 = N \frac{t_{mux}}{2t_{carry}}$$

$$K_{opt} = \sqrt{N \frac{t_{mux}}{2t_{carry}}}$$

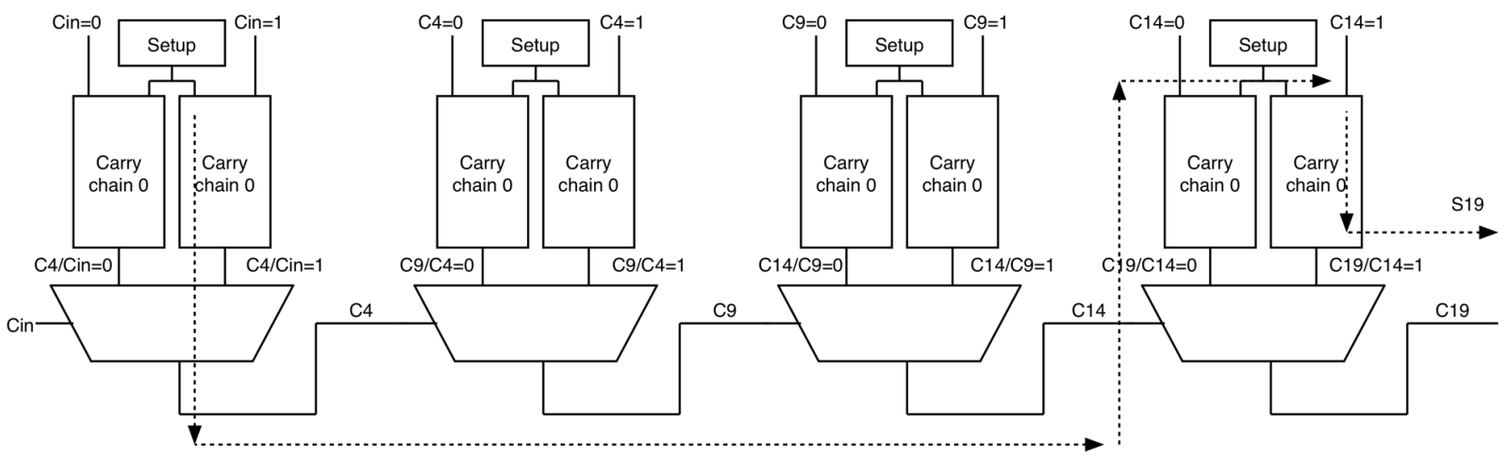
3B. ADDERS PART 2

CARRY-SAVE ADDER



- Sacrifice area for delay
- Each block calculates assuming either value for Cir
- All Couts ready once the first is ready
  - Only wait for one final propagate through MUX'es

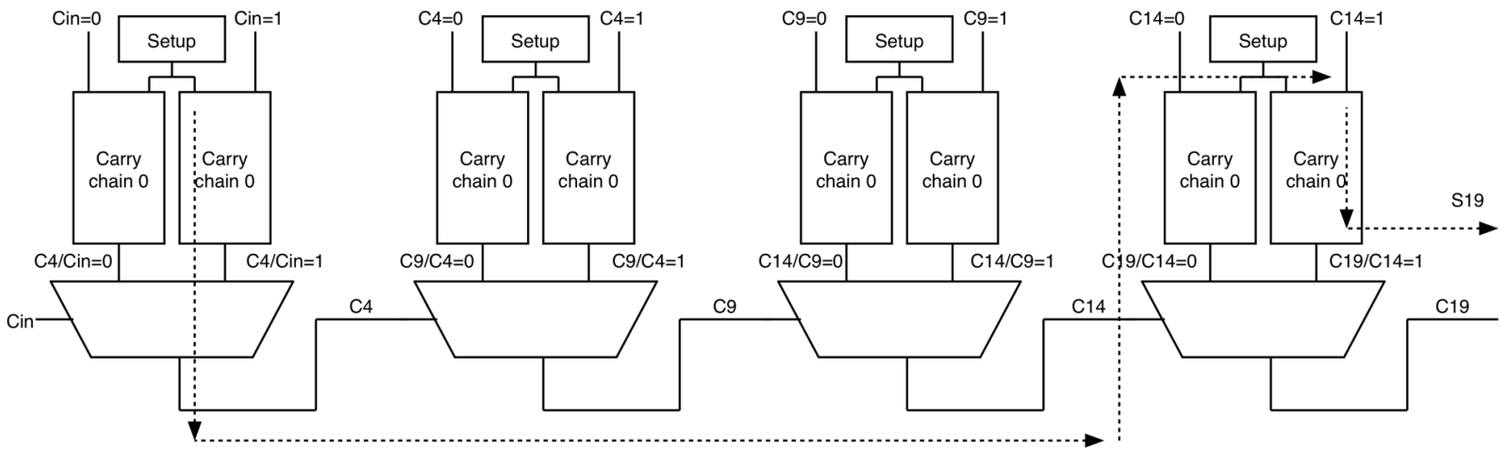
CARRY-SAVE ADDER



- Similar to bypass adder
- Both remain linear with impact only on slope

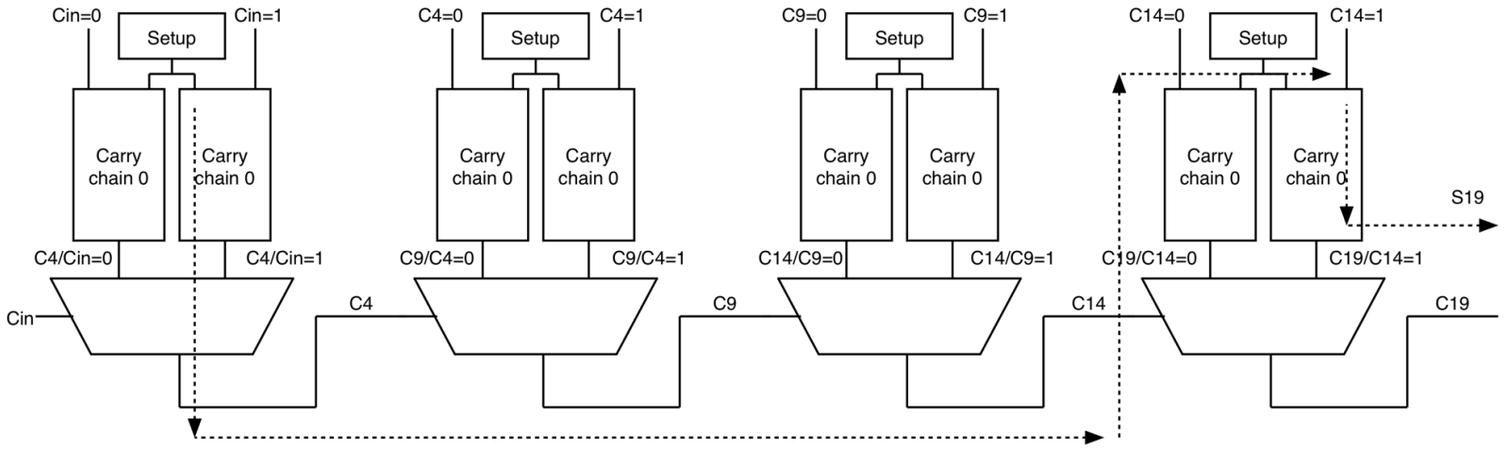
$$t_{pd} = Kt_{carry} + (N/K - 2)t_{mux} + (K - 1)t_{carry} + t_{sum}$$

WHY IS DELAY LINEAR?



Carry	Delay to calculate carry	Delay to multiplex carries
Cout4	5Tcarry	NA
Cout9/0 Cout9/1	5Tcarry	5Tcarry+Tmux
Cout14/0 Cout14/1	5Tcarry	5Tcarry+2Tmux
Cout19	5Tcarry+2Tmux+5Tcarry	NA

SUB-LINEAR ADDERS THROUGH UNEQUAL BLOCKS



$$N = K + (K+1) + (K+2) + \ldots + (K+P-1)$$

$$N = P(K+K+P-1)/2 \qquad N \approx P^2/2$$

$$t_{pd} = Kt_{carry} + (P-2)t_{mux} + (K+P-2)t_{carry} + t_{sum}$$

$$t_{pd} = Kt_{carry} + (\sqrt{2N}-2)t_{mux} + (K+\sqrt{2N}-2)t_{carry} + t_{sum}$$

WHAT IS AN IDEAL ADDER?

- An ideal adder has N-independent delay
- Consider:
  - $C0 = G0 + P0\ Cin$
  - $C1 = G1 + P1\ C0$
  - $C2 = G2 + P2\ C1$
  - $C3 = G3 + P3\ C2$
  - $C4 = G4 + P4\ C3$

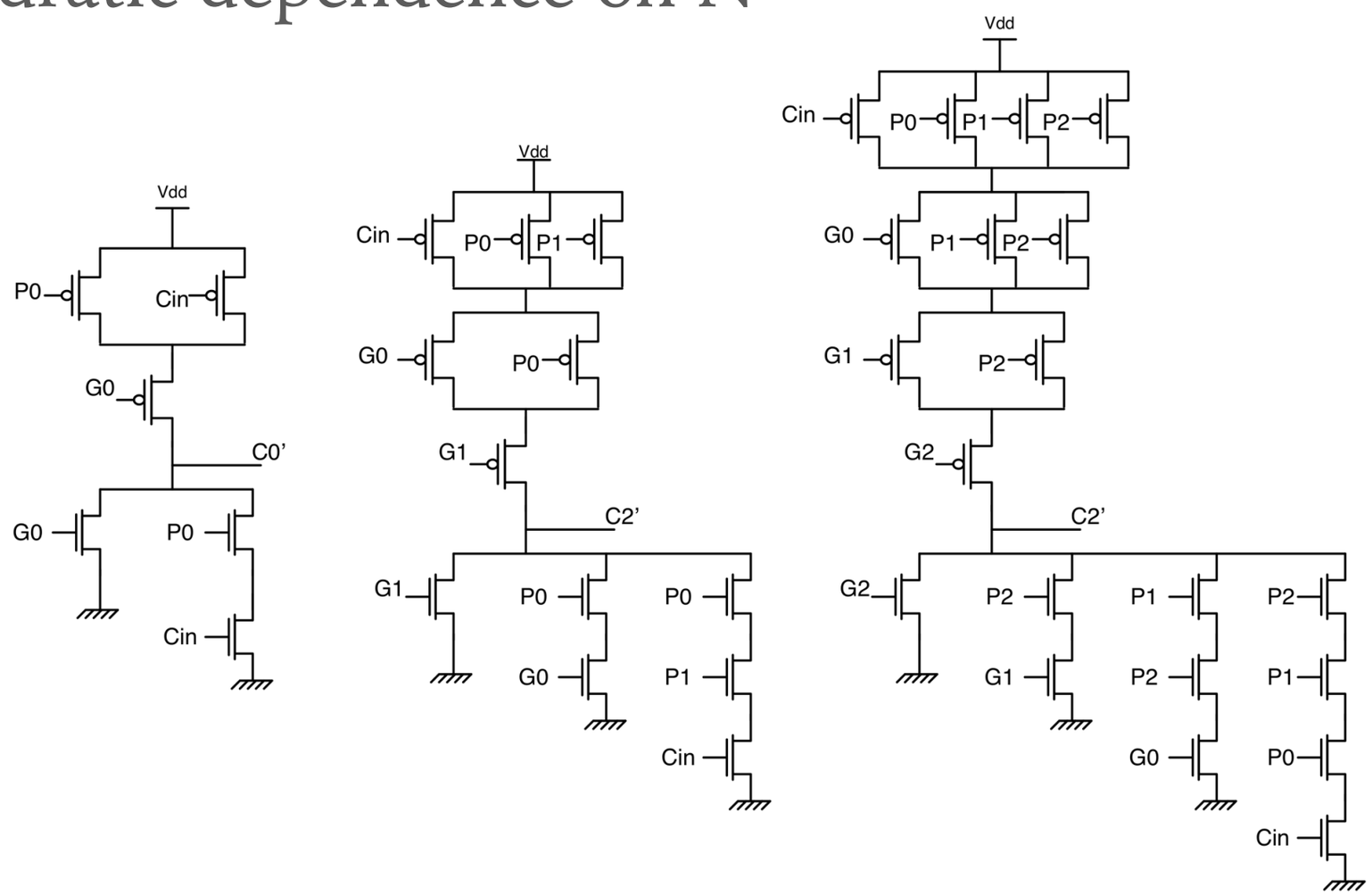
BUT ALL CARRIES ARE EXPANDABLE

- We can expand carry expressions
- Examining them, they all depend only on Cir and G's, P's
- All can start to be calculated immediately
  - Did we find an adder independent of N?

$$C_0 = G_0 + P_0 C_{in}$$
$$C_1 = G_1 + P_1 G_0 + P_0 P_1 C_{in}$$
$$C_2 = G_2 + P_2 G_1 + P_1 P_2 G_0 + P_0 P_1 P_2 C_{in}$$
$$C_3 = G_3 + P_3 G_2 + P_2 P_3 G_1 + P_1 P_2 P_3 G_0 + P_0 P_1 P_2 P_3 C_{in}$$
$$C_4 = G_4 + P_4 G_3 + P_3 P_4 G_2 + P_2 P_3 P_4 G_1 + P_1 P_2 P_3 P_4 G_0 + P_0 P_1 P_2 P_3 P_4 C_{in}$$

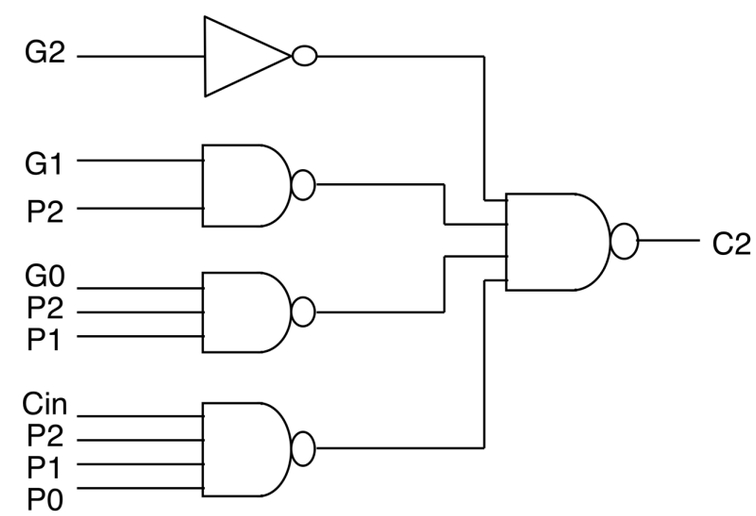
WE ARE COMPLETELY OFF

- Longer final chains, more connected branches
- Increase in both R and C
  - Quadratic dependence on N



ANOTHER WAY TO LOOK AT LOOKAHEAD ADDERS

- Two tiered logic
- Dominated by the final NAND gate
- Size of NAND gate increases with N
  - Electrical effort is dependent on N
  - Delay is O(N)



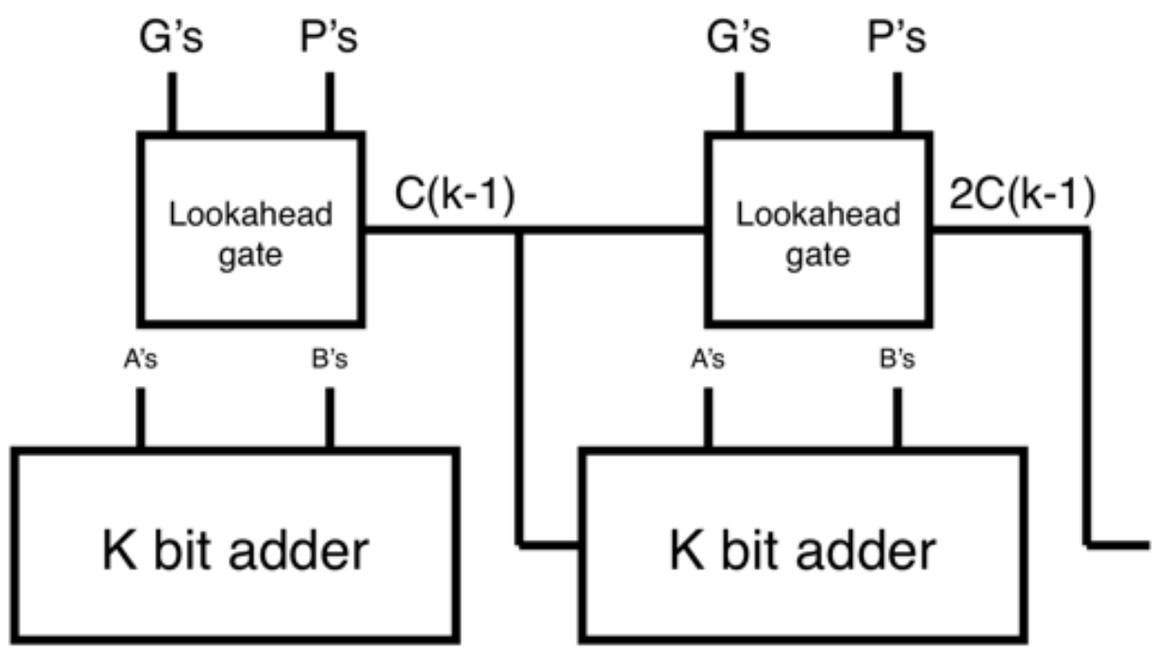
CONCLUSIONS ABOUT LOOKAHEAD ADDERS

- At worst, the dependence is quadratic
- At best it is linear
- Thus it cannot be any better than ripple carry adder

NOT SO FAST

- Lookahead adders have bad scaling properties
  - However, because lookahead gates are rather simple they have good y-intercepts
  - Thus for very small N, the lookahead gate can be useful

MAKING USE OF LOOKAHEAD ADDERS



- Use one lookahead gate to calculate final bit position
- The lookahead gate better be simple
  - Best use a Manchester chain

ANALYZING A CARRY EXPRESSION

$$C_1 = G_1 + P_1 G_0 + P_0 P_1 C_{in}$$

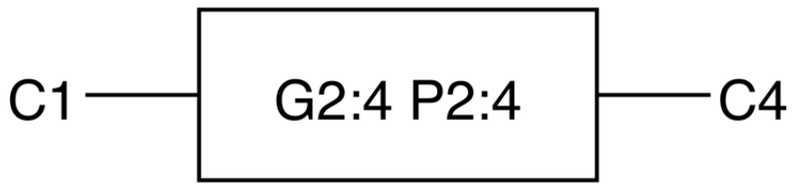
- The first term is when a carry is generated in the current position
- The second is when a carry is generated in the previous position and propagated to the current position
- The final term is when a carry in exists to the range and every position is propagating

DETECTING A PATTERN

$$C_2 = G_2 + P_2G_1 + P_1P_2G_0 + P_0P_1P_2C_{in}$$

- First term is when a carry is generated in the current position
- The two middle terms are:
  - Carry generated anywhere in the range and propagated to current position
- Final term is carry in to the range propagated throughout

GROUP G'S AND P'S



- The range 2:4
- G2:4 means:
  - A bit is generated anywhere in the range and propagates to its output
- P2:4 means:
  - All positions in the range are propagating

CALCULATING GROUP G'S AND P'S

$$G_{3:6} = G_6 + G_5P_6 + G_4P_5P_6 + G_3P_4P_5P_6$$
$$P_{3:6} = P_3P_4P_5P_6$$

- G3:6 can be calculated by induction or by observing the definition
- Same for P3:6
- The value of groups is that they allow hierarchies and fast adders
  - Must progressively calculate group properties

THE DOT OPERATOR

$$(G_{x:y}, P_{x:y}) \circ (G_{y+1:z}, P_{y+1:z}) = (G_{x:z}, P_{x:z})$$

- Combines two groups into a larger group
- Actually two operations
- Order matters

$$(G_{x:y}, P_{x:y}) \circ (G_{y+1:z}, P_{y+1:z}) = (G_{y+1:z} + G_{x:y}P_{y+1:z}, P_{x:y}P_{y+1:z})$$

QUESTIONS ABOUT THE DOT OPERATOR

- Is it commutative?
- What happens if it acts on ranges with overlap?
- What happens if it acts on ranges with gaps?
- When do we consider the adder “done”?

IS THE DOT OPERATOR COMMUTATIVE?

- No
- Obviously it is not because the calculation of group generate requires order of operands
  - We must know which range is an “input” to the other

$$(G_{x:y}, P_{x:y}) \circ (G_{y+1:z}, P_{y+1:z}) = (G_{y+1:z} + G_{x:y}P_{y+1:z}, P_{x:y}P_{y+1:z})$$

WHAT HAPPENS IF THE DOT OPERATOR ACTS ON OVERLAPS

- Overlaps are ignored
- You can safely combine ranges with overlaps
- This can be easily proved

WHAT HAPPENS IF IT ACTS ON A RANGE WITH GAPS

- The result is meaningless
- If this happens at any point there is something wrong with the design

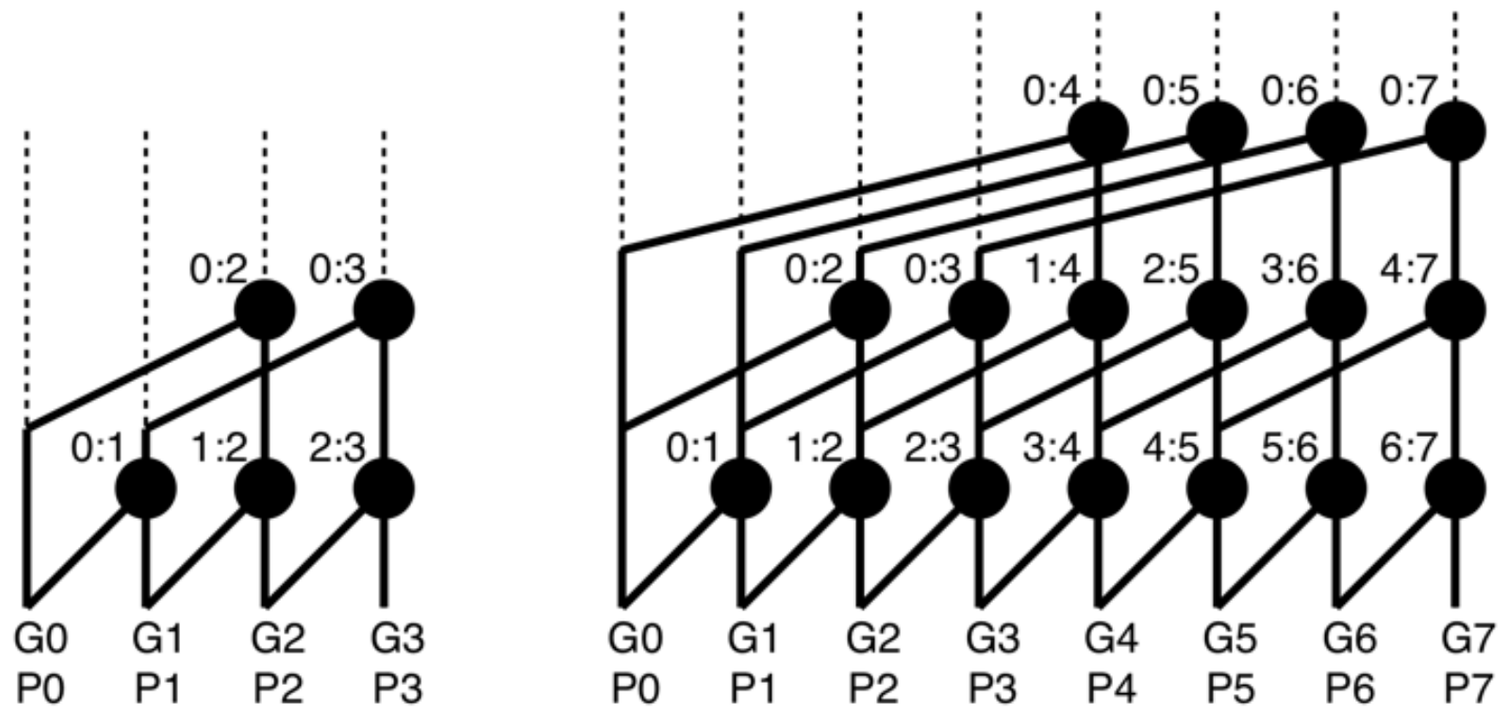
WHEN IS IT DONE?

- We can consider ourselves done when we have the group properties for all positions
- Group properties means the carry is ready
  - We just need to multiply the carry in with the group propagate

PARALLEL PREFIX ADDERS

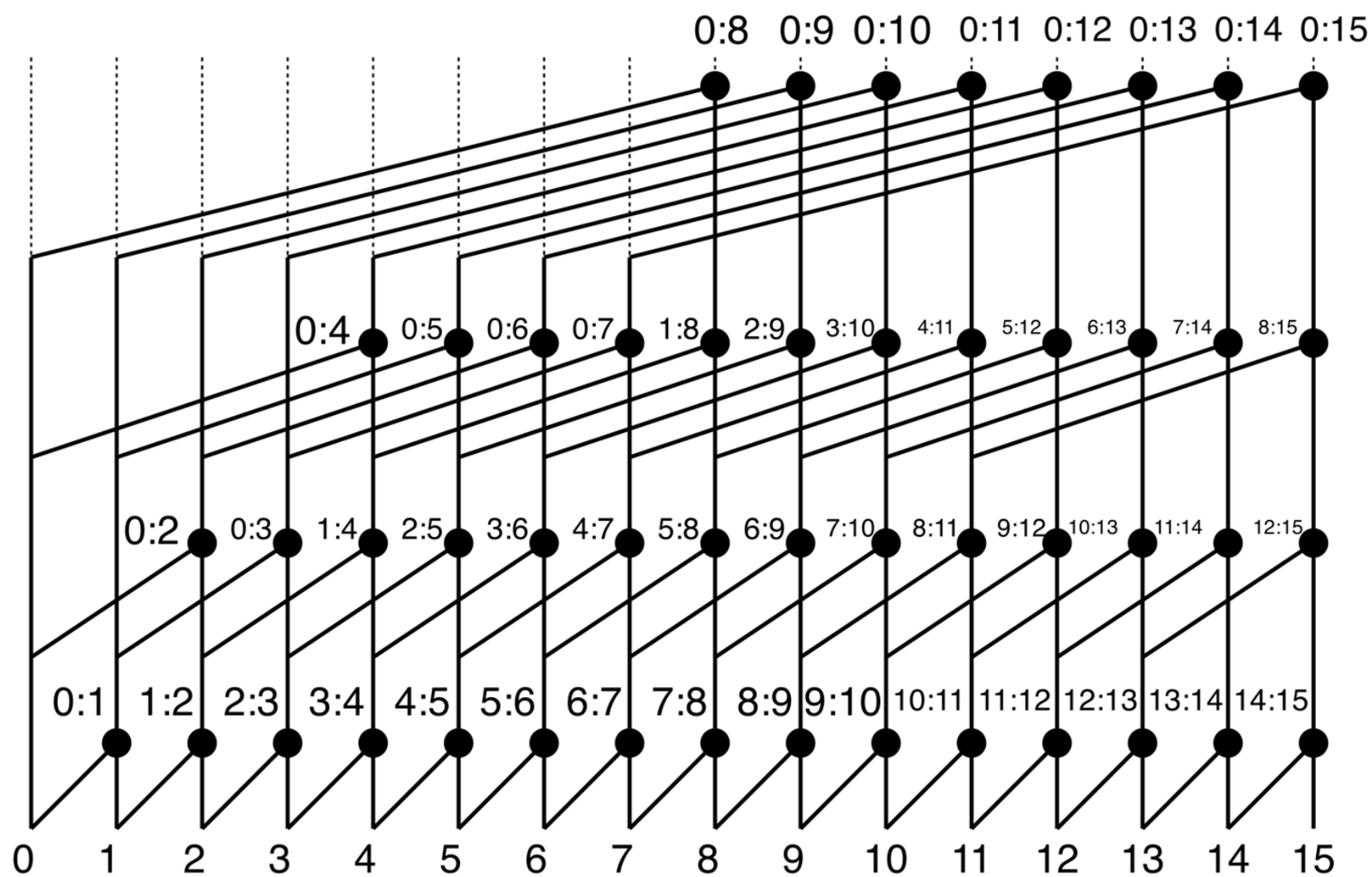
- Offer some of the fastest possible adders
- As the name suggests parallelism is the main method of achieving speed
  - This is at the cost of area
  - Routing can also become very difficult

KOGGE STONE ADDERS



- Systematic architecture
- Use smaller group properties to form larger properties

K-S DELAY



- The number of consecutive dot operators increases logarithmically

K-S ADDERS ADVANTAGES AND DISADVANTAGES

- Advantages
  - Great delay scalability O(log2(N))
  - Fanout is under control (2 per operator)
- Disadvantages
  - Devastating routing requirements
  - Area grows very quickly

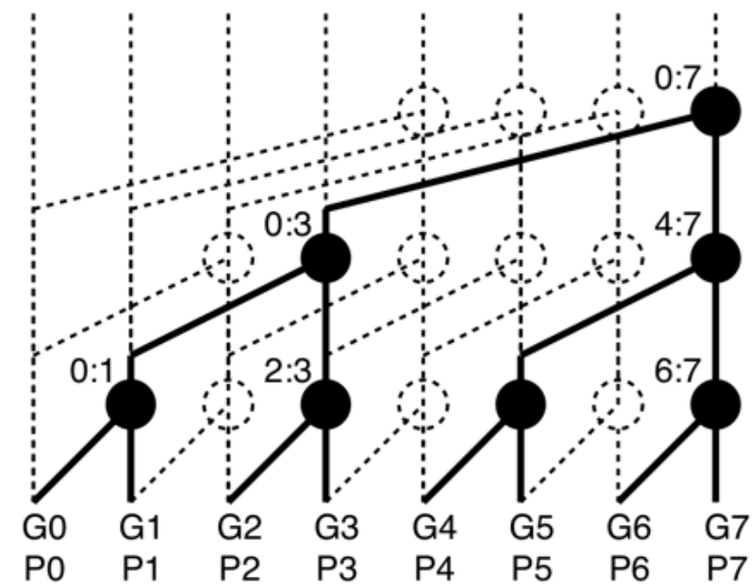
$$\sum_{i=1}^{\log_2(N)} (2^N - 1) - 2^{(i-1)}$$

WHEN IS THE ADDER "DONE"?

- We are focused on the last carry position
  - Because it is the last to be finished
- But We actually need every carry position
- The ultimate aim is to calculate all the sum bits
  - To calculate the sum bits we need the carry bits
- Thus we need **all** the carries to consider ourselves done
  - And the last carry controls critical path

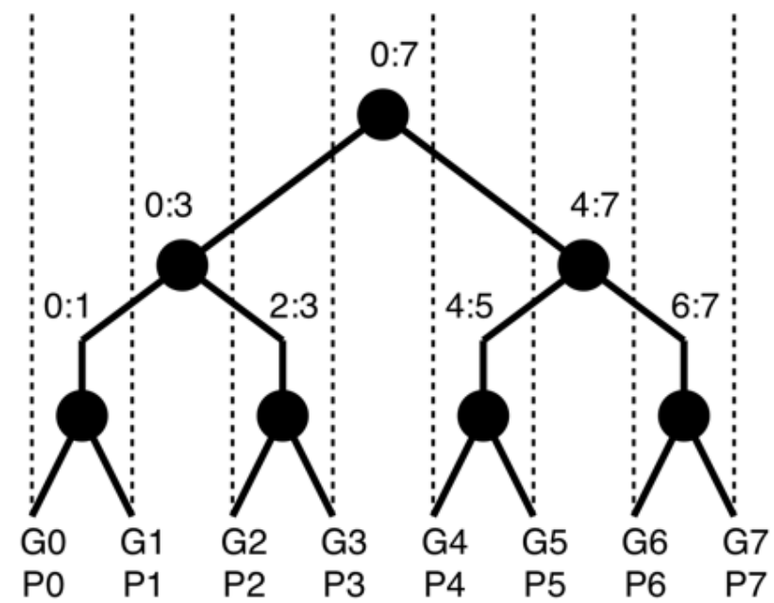
WHAT IF WE ONLY NEEDED THE LAST CARRY

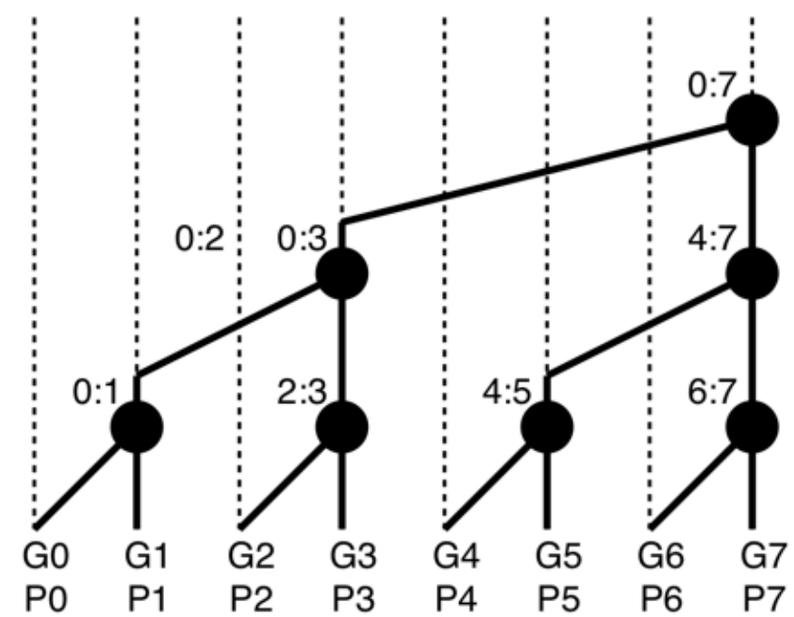
- We can trim much of the tree
- But we must be aware that this is not an N-bit adder
- We can only calculate the last bit of the output



CALCULATING ONLY THE LAST CARRY

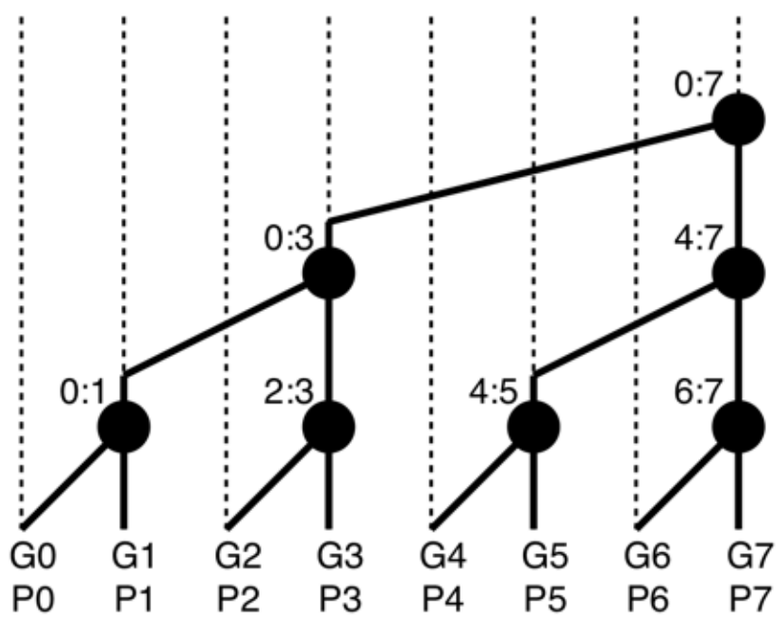
- The tree can be deduced from scratch
- Combine bit positions in pairs
- The logarithmic growth of the critical path is now clear

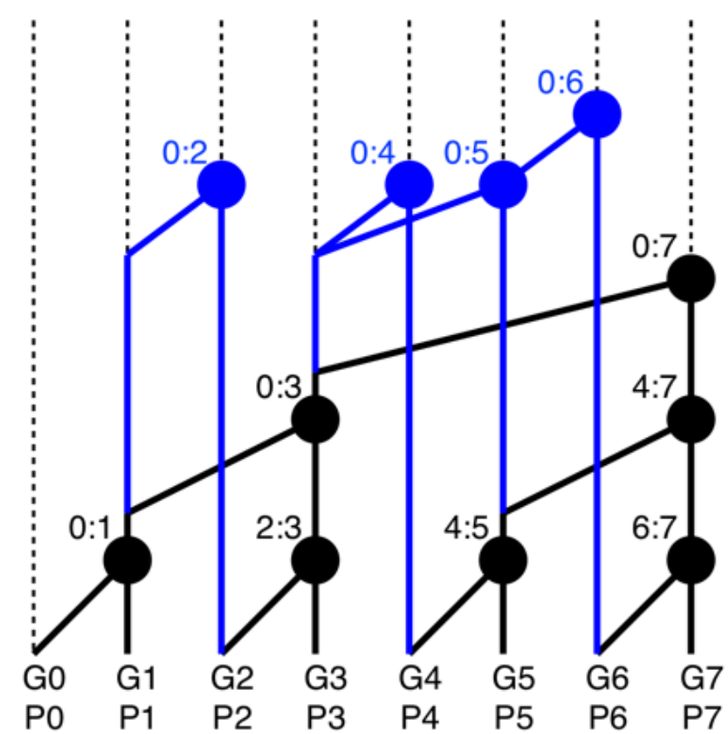




COMPLETING THE ADDER

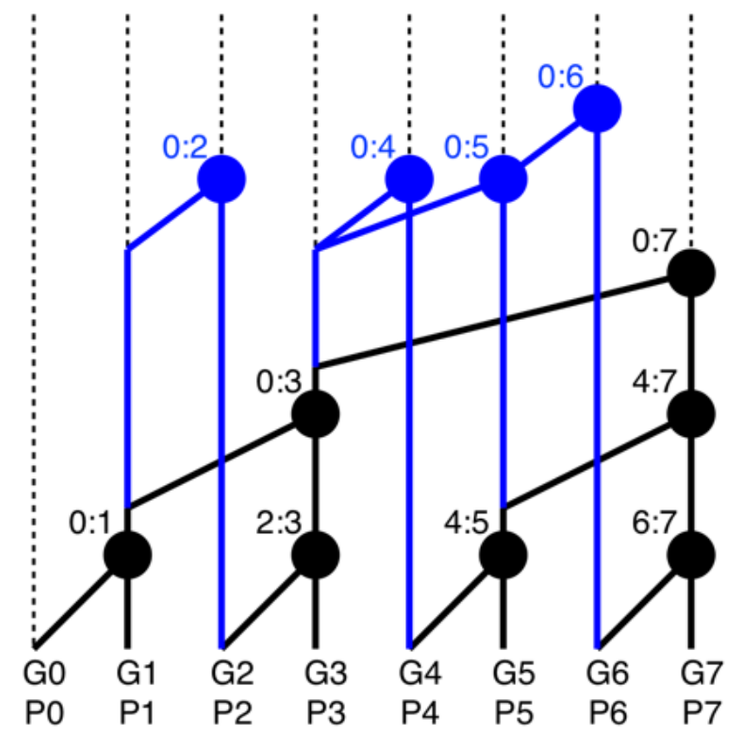
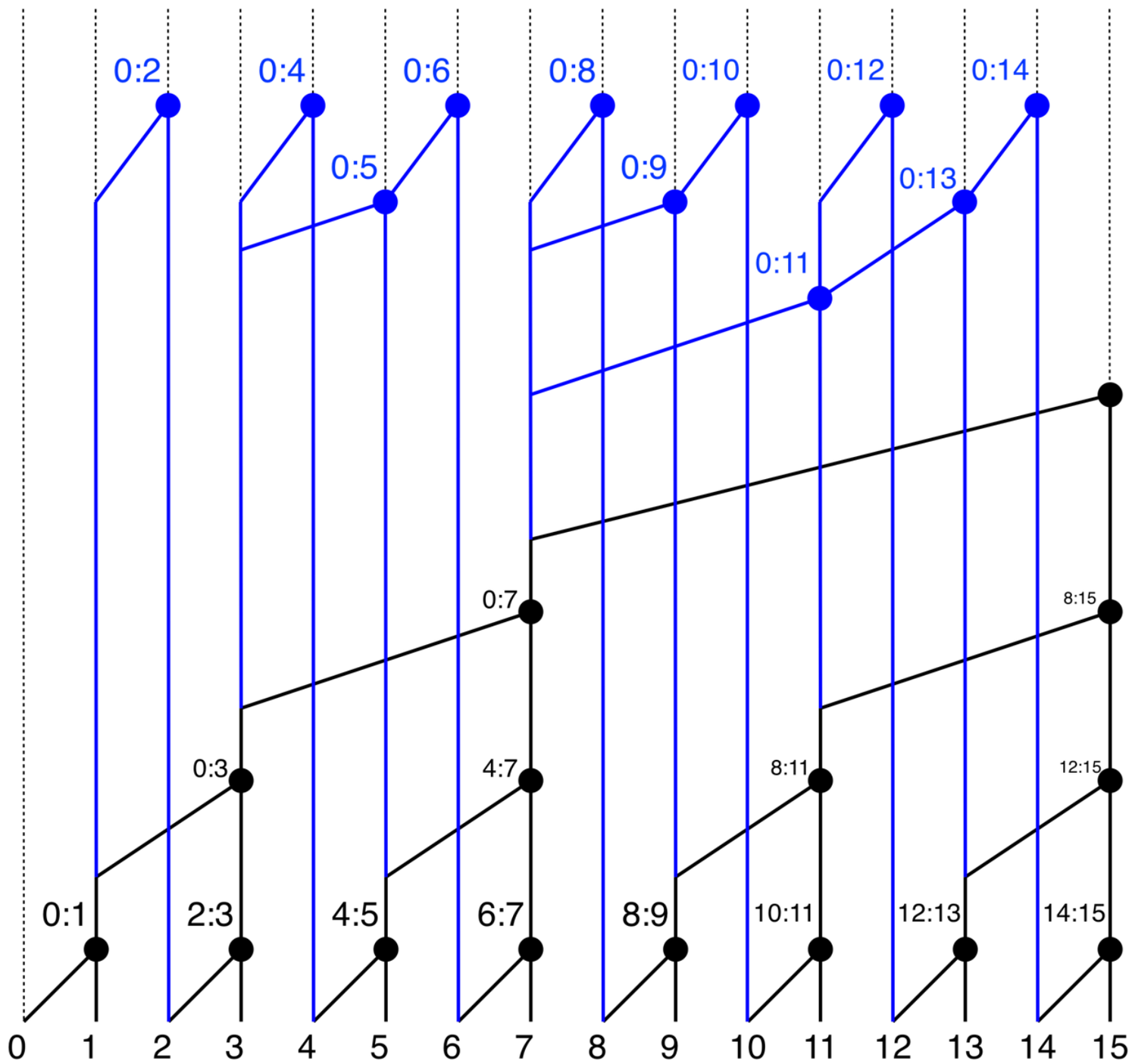
- The C7 adder is missing a lot to be a true adder
- But we can manually form the missing carries
  - Notice some of the carries are originally obtained by "accident"





BRENT-KUNG ADDERS

- BK adders are alternative parallel prefix adders
- Delay grows slightly faster than KS adders
- Area is smaller
- Routing is much easier
- Fanout is increased



16 BIT B-K ADDER